

Deep Reinforcement Learning for Bitcoin Trading

Bouchra El Akraoui[✉] and Cherki Daoui[✉]

Laboratory of Information Processing and Decision Support,
Sultan Moulay Slimane University,
B.P. 523, Beni-Mellal, Morocco
mbouchra2@gmail.com, daouic@yahoo.com

Abstract. Artificial intelligence (AI) is showing its success in various types of applications. Motivated by this trend, automatic trading has given new attention to the application of artificial intelligence techniques to predict the future price of a financial asset to overcome trading challenges including asset price fluctuations and dynamics, so the investors need to understand when it is appropriate to buy or sell the asset or hold the current position to maximize return on their investment. However, it is difficult to get a perfect strategy in a complex and dynamic price asset. To address these issues, in this paper, we propose a new rule-based policy approach to train one of the machine learning algorithms, called Deep Reinforcement Learning (DRL) to trade Bitcoin. The proposed method is based on Dueling Double Deep Q-learning Networks (DD-DQNs), Proximal Policy Optimization (PPO), and Advantage Actor-Critic (A2C) to achieve an optimal policy. The Sharpe ratio and profit reward functions are used to evaluate the proposed DRL. The experimental results show that the integration of three agents is the most profitable approach for trading Bitcoin.

Keywords: Markov decision process · Deep Reinforcement Learning · Dueling Double Deep Q-learning Networks · Proximal Policy Optimization, Advantage Actor-Critic

1 Introduction

Cryptocurrencies have gained a great deal of attention from researchers in the previous few years due to the experienced massive growth in Bitcoin prices, resulting in a series of traders' opportunities to speculate on market prices, expanding research on financial trading, and suggesting a variety of methods for trading in the markets. One of the most difficult aspects of automatic trading is predicting the best strategies to maximize rewards. It is difficult to devise a profitable strategy within a dynamic and complicated price of a financial asset. Especially since bitcoin price fluctuations are erratic and are more volatile than other major asset classes, which makes them difficult to predict. Therefore, the investors need to understand the appropriate time to buy or sell the asset or

hold the current position to maximize the return on their investment. Price prediction theory is the decision-making process that is one of the main topics of discussion in finance that aims to maximize return while minimizing risk. One of the classical methods is carried out in two stages as mentioned in [22]. The stock prices' covariance matrix and predicted returns are computed first. The optimal asset allocation is therefore determined by either maximizing profit for a given investment risk or avoiding risks for a variety of returns. Following the optimal asset allocation, the profitable trading strategy is subsequently discovered. However, if the manager wants to alter the decisions taken at each time step and take into account factors such as transaction cost, this strategy might be extremely difficult to implement. For this purpose, the agent requires more than a price prediction model [11]. Several methods have been proposed to address the issue of automated trading, especially, when the data had become available and market information had become more accessible [8]. The most efficient procedures that are based on optimization require deeper knowledge [12]. Another approach to tackling the challenge of cryptocurrency trading is to represent it as a Markov Decision Process (MDP) and one of the most effective methods to find the best strategy to solve these problems is Reinforcement Learning (RL). However, it is now a pretty common tool for decision-making planning and learning, as well as determining optimal strategies in a partially known or unknown system. An MDP is a frequent means of simulating such a situation. The current state in the financial environment could be the actual price, current return, as well as some forecasting index values, and the next state he or she will end up in after taking the action are all provided in an MDP. The goal of RL from a financial standpoint is to determine the optimal strategy for a given situation that maximizes the expected discounted reward over a set of states. Though, the scalability of the RL model is constrained due to the enormous state spaces [27] [9] [?, 1]. With the advent and tremendous progress of Deep Learning (DL), we have been able to create a variety of algorithms capable of fully automating complex tasks such as the recognition of images and classification [21]. Motivated by the mentioned challenges, we present in this paper, a recently successful and outstanding combination of the above methods (RL and DL frameworks), to generate automatic trading systems using a new research route named Deep Reinforcement Learning (DRL), which recommends taking the right action and achieves tremendous performance in trading by outperforming the traditional methods to maximize the rewards and get the best result in the complex, dynamic and high dimensional financial environment. This innovative method is based on the famous Deep Q-Network (DQN) algorithm and has been adapted to the specific challenge of algorithmic trading. In trading financial models, the proposed technique faces the following challenges:

- i) Dynamic systems: the proposed models must be able to adjust to market changes fast. Even though financial markets are so volatile, and to cope with non-linear models of financial markets;

- ii)* The models must be flexible to every asset in diverse markets: Using the history of asset-specific price data, the suggested models must be able to learn appropriate trading strategies for asset classes in diverse economies
- iii)* Feature extraction: The feature extraction phase is one of the most significant elements of the suggested models for learning a decent trading rule for a certain financial asset. The performance of the learned trading rules is directly influenced by the quality of the extracted characteristics
- iv)* Risk management: It is a key element of the financial market, which is afflicted by price fluctuations.

The remainder of the paper proceeds as follows. Section 2 provides a discussion of relevant topics in the related literature. Section 3 describes the methodological background contains a brief overview of RL methods. Section 4 describes the proposed DRL approach. Section 5 presents the results of the experiments along with a discussion of the most relevant results. Section 6 presents the conclusions.

2 Related Works

Since the beginning of trading, traders have sought to maximize the investor's return by using all sorts of methods and techniques including technical analysis [22] [11] [8] fundamental analysis [28], and algorithmic trading [27] [9] [1] [31], but only some of them have succeeded. Over the past decade, artificial intelligence (AI) has attracted increasing interest from the scientific community, changing many areas. A key element for this expanding attention is related to the dramatic successes of deep reinforcement learning techniques. However, little research focuses on the application of artificial intelligence to cryptocurrency trading strategies. Outstanding trading strategies cannot only assist investors to achieve high returns but also help them reduce revenue risk effectively [29]. In this context, some related articles are studies of cryptocurrency trading strategies engaged in developing efficient models for predicting the direction of cryptocurrency prices and then designing specific trading strategies to assist investors in taking advantage of them. Most of them use supervised learning algorithms to predict Bitcoin price movement as [23], [3] [17] [10]. They have applied various Machine Learning (ML) algorithms likewise Logistic Regression, Naive Bayes, Support Vector Machines, and random forests to obtain a high accuracy of around 98.70 %. With the advent of DL algorithms and their notable success in several fields, notably in image processing [2], [16] [4] have benefited from this eventful realization to obtain good results based on Deep Neural Networks (DNNs). In the same way, the most recent research [5] [36] [38] focus on the application of DL to financial markets to find the best results. Although DL techniques have gained considerable performance in this area, they have many shortcomings, including computational and numerical limitations, sub-optimal optimization of parameters, and poor generalization [37]. More recently, Huang Chien-Yi et al [15] provided a Markov Decision Process (MDP) model adapted to the financial trading problem using the Deep Recurrent Q-Network (DRQN)

algorithm that achieves favorable outcomes in many of the simulation parameters [29]. According to these techniques, several studies have already investigated the Reinforcement Learning (RL) methods to tackle this algorithmic trading problem with high-quality results. For instance, Azhikodan et al. in [6] aim to validate that RL can successfully learn the tricks of financial trading. Similarly, Hegazy and Mumford [14], in their comparative study showed the best performance of the Recurrent Reinforcement Learning algorithm (RRL). Despite the successes of RL [7], these findings are unable to handle high-dimensional issues due to the lack of scalability [26], [19]. In another effort to objectively evaluate the performance of trading strategies, this paper proposes a more rigorous novel methodology for evaluating trading performance using DRL who is the newest AI technology that has high power to solve the limitations of traditional DL approaches. DRL interacts with the task in a sequential manner, allowing it to progressively understand the task’s intrinsic knowledge and then accomplish it efficiently. This family of methods focuses on the learning procedure of an intelligent agent (i) addressing difficult sequential decision-making problems. (ii) interpreting sequentially with an unknown environment (iii) maximizing its cumulative rewards and using DL techniques to generalize the information learned from acting with the environment [32]. (iv) Processing complex, nonlinear models of economic data effectively surmounts these challenges [13]. DRL brings a novel and powerful AI-based guideline that is getting more popular in autonomous decision-making and operation control by combining the power of DL in solving most of the large and challenging problems with RL, which has a generic, flexible, and comprehensive tool for sequential decision-making problems. To solve the aforementioned issues, this study proposes a new rule-based policy based on DRL by implementing Dueling Double Deep Q-Networks (DD-DQNs), Proximal Policy Optimization (PPO), and Advantage Actor-Critic (A2C) to achieve an optimal policy. For more details, a brief overview of RL and DRL is introduced in the next section.

3 Methodological backgrounds

In this section, we will briefly describe what deep reinforcement learning is and we will present some popular DRL algorithms that have been used in trading cryptocurrency.

3.1 Reinforcement Learning

RL is a branch of machine learning that employs a trial-and-error method of learning. This differs from other machines in that RL is a powerful learning strategy that does not require labeled data. The topic has seen a recent renaissance with the successful demonstration of the use of deep learning in RL to solve real-world tasks, where DNNs are employed to make decisions, as mentioned above DRL is a combination of DL and RL and it is the focus of this work. Researchers have recently used DRL and shown success in many studies,

especially, in the stock markets to make portfolio bets, they have outperformed traditional cryptocurrencies buying and holding strategies [18]. Reinforcement learning involves formalizing an optimal strategy for ensuring the maximization of the expected cumulative benefit. When interacting in the environment, the RL agent gains rewards and observations. When the trading agent is in state $s_t \in S$, it must choose an action $a_t \in A$ in at each discrete timestamp t whether to buy or sell the asset being considered or to hold the current position the trading system based upon the decision rule described in Equation 1, where S stands the set of possible states, and A refers to the set of controllable actions, determining which actions to take is based on a policy π which describes the agent's behavior and tells it which actions to select for each possible state. As a result of each action $a_t \in A$, a scalar reward $r_t \in R$ is given to the agent, and the next state $s_{t+1} \in S$ is observed. The probability of transition of each possible next state s_{t+1} is made to be $\Pr(s_{t+1}|s_t, a_t)$, where s_{t+1} . Likewise, the reward probability of each possible reward r_t is defined as $\Pr(r_t|s_t, a_t)$. Therefore, the expected scalar reward, r_t , received by acting a in current state s_t is computed based on $E_{Pr(r_t|s_t, a_t)}(r_t|s_t = s, a_t = a)$. The agent's interaction with the environment results in a trajectory $\tau = [s_0, a_0, r_0, s_1, a_1, r_1, \dots]$.

$$Action(A) = \begin{cases} 2, & \text{if } E[pr_{t+n}] > pr_t \\ 1, & \text{if } E[pr_{t+n}] < pr_t \\ 0, & \text{if } E[pr_{t+n}] = pr_t \end{cases} \quad (1)$$

According to the asset prices, pr_t at time t , the automated trading process choose an action 1 (i.e Sell) if the expected price at time $t + n$ (i.e $E[pr_{t+n}]$) is lower than pr_t , and when is greater than pr_t the agent takes an action 2 corresponding to the sell, otherwise it does nothing do (hold, i.e 0). The number n is referred to as the future time steps.

Stop-loss (S_l) and take-profit (T_p) orders are two more orders that are frequently utilized in creating trading strategies. When an asset reaches a certain price, both of them are used to buy or sell it. Stop-loss orders are intended to limit the amount of money that can be lost. Take-profit is a strategy for ensuring a profit. The DRL process is illustrated in figure 1, and this framework can be modeled as finite MDPs.

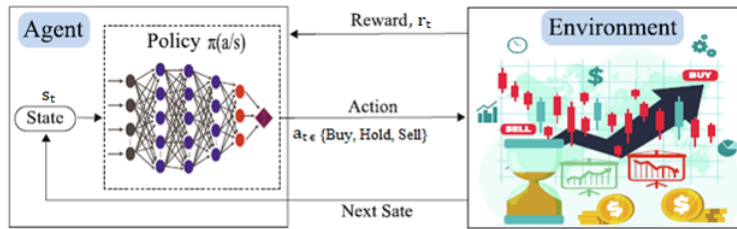


Fig. 1. The DRL framework

The learning agent's goal is to determine an optimal policy π^* , which describes the probability of taking action a_t in state s_t in order to maximize the sum of discounted rewards over time. The following is the formula calculates the expected discounted return R at time t :

$$R_t = \sum_{k=t+1}^T \gamma^{k-t-1} r_k \quad (2)$$

Where T is the planning horizon, it can be ∞ and $\gamma \in [0, 1]$ is the discounting rate. The discount factor γ determines the relevance of future rewards while emphasizing the importance of recent rewards. A greater discount rate value in algorithmic trading indicates that the agent is more long-term investment-oriented. In the case of $\gamma = 0$. Only the immediate reward has an effect. The agent analyzes each reward fairly during the trade in the ultimate scenario of $\gamma = 1$. If we let $\gamma \rightarrow 1$, future benefits are taken into account more heavily. This key parameter is adjusted according to achieve the desired result. The state-value function, $V_\pi : S \rightarrow \mathbb{R}$ attributes a value to each state that follows a policy π . $V_\pi(s_t)$ is the expected return value when starting in state s_t and thereafter following policy π . For MDPs, we can formally define $v_\pi(s_t)$ as.

$$V_\pi(s_t) = E_\pi[R_t | S_t] = E_\pi \left[\sum_{k=t+1}^T \gamma^{k-t-1} r_k | S_t \right] \quad (3)$$

Similarly, the action-value function or Q-value, $Q_\pi : S \times A \rightarrow \mathbb{R}$ denoted $Q_\pi(s_t, a_t)$, formally defined by equation (3), gives the expected return when the environment starts from s_t , taking the action a_t , and follows policy π .

$$Q_\pi(s_t) = E_\pi[R_t | S_t, A_t] = E_\pi \left[\sum_{k=t+1}^T \gamma^{k-t-1} r_k | S_t, A_t \right] \quad (4)$$

The objective of reinforcement learning is to find a policy that generates a large amount of reward over time. We can exactly define an optimal strategy for finite MDPs in the following manner. A partial ordering of policies is defined by value functions. If a policy's expected return π is larger than or equal to π_0 for all states, it is said to be better than or equal to policy π_0 . At least one policy is always better than or equal to all others. This is called an optimal policy. Although there may be more than one, we refer to all optimal policies. They have the same state-value function, which is known as the optimal state-value function and is symbolized by V^* as follows:

$$V_\pi^*(s_t) = \max_\pi V_\pi(s_t), \text{ for all } s_t \in S.$$

Similarly, the optimal action-value function denoted Q^* and is defined as:

$$Q_\pi^*(s_t, a_t) = \max_\pi Q_\pi(s_t, a_t)$$

The action-value function can be redefined according to Bellman’s optimality equation as follow:

$$Q_{\pi}^*(s_t, a_t) = r + \gamma \max_{a_{t+1}} Q_{\pi}(s_{t+1}, a_{t+1}) \quad (5)$$

The Q-learning algorithm is one of the most frequently used value-based RL algorithms [34], the agent updates the Q-value of the current state s_t considering the optimal Q-value of the next state $Q_{\pi}^*(s_{t+1}, a_{t+1})$ where action a_{t+1} is a greedy action based on a Q-matrix that is updated at each time step according to Bellman’s optimality equation as shown in the following equation:

$$Q_{\text{new}}(s, a) \leftarrow (1 - \alpha)Q_{\text{old}}(s, a) + \alpha \left[r + \gamma \max_{a' \in A} Q_*(s', a') \right] \quad (6)$$

Under the assumptions that: the state-action pairings typically described discretely, as well as any actions are repetitively sampled throughout the states to guarantee enough exploration, and thus not required access to the transition model, one generic demonstration of convergence to the optimal value function is known [34]. A parametric value function $Q(s, a, \theta)$ is required in this case, where θ refers to the parameters that specify the Q-values. In the literature, there are several different Q-networks like Deep Q-Networks (DQNs) [25] and Double Deep Q-Networks (D-DQNs). The proposed Q-learning trading system is based on Dueling Double Deep Q-Networks (DD-DQNs) [33], motivated by the successful outcome in [20], the DD-DQNs use a dueling network design to estimate the value function $V(s)$ and the corresponding advantage function $A(s, a) = Q(s, a) - V(s)$ and then combine these to estimate $Q(s, a)$. In DD-DQN, a Convolutional Neural Networks (CNNs) layer is followed by two streams of fully connected (FC) layers, which are used to estimate the value function and the advantage function separately before combining the two streams to estimate the action-value function.

In the next section, we introduce our setups including state, action spaces, and reward functions. We describe three RL algorithms used in our work, DD-DQN, and both advanced policy gradient approaches are employed as A2C and PPO methods.

4 Proposed Deep Reinforcement Learning Module

DD-DQN is the foundation of the planned Q-learning trading system. An agent interacts with the financial market in this situation. The agent determines the type of action to take on a bitcoin unit based on the current status of the financial market. After acquiring a bitcoin, it is stored in a wallet. A stop-loss ($S_l = 5\%$) and a take-profit ($Tp = +15\%$) are indeed implemented to the wallet. For example, if the wallet loses more than a certain amount (for example, 5%), all open positions are closed. For deep RL techniques as well as the proposed Q-learning trading system, the exploration-exploitation issue is critical. Exploitation is concerned with environmental information (i.e. transition

and reward functions), whereas exploitation is concerned with maximizing profit. As a result, the agent can perform a random action with probability, ϵ , and then follow the optimal policy with probability, $1 - \epsilon$ (ϵ -greedy technique). For the first observations ($n_{obs} = 500$) in the proposed trading system, an initial value of $\epsilon_i = 1$ is chosen, and then a new ϵ value $\epsilon_{new} = 0.15$ is established. A trade transition cost of 0.3 % is used for both long and short actions to make the analysis more realistic.

4.1 Advantage Actor-Critic

The Advantage Actor-Critic policy gradient method is an asynchronous version of the Asynchronous Actor-Critic (A3C) policy gradient method. In comparison to the asynchronous implementation of A3C, synchronously changes parameters to increase GPU efficiency [24]. A2C is a deterministic, synchronous The A2C (Advantage Actor-Critic) policy gradient method is an asynchronous version of the A3C policy gradient method. In comparison to the asynchronous design of A3C, and synchronously changes parameters to increase GPU efficiency [24]. It is better able to adapt to risk [35]. A2C is a deterministic, synchronous implementation that allows for each actor to complete their section of experience before adjusting and aggregating across all actors. Because of the bigger batch sizes, GPUs are used more effectively. The A2C update is determined using the following formula:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta'} \log \pi(a_t | s_t; \theta') A(s_t, a_t; \theta, \theta_v) \quad (7)$$

Where the estimate of the advantage function denoted by $A(s_t, a_t; \theta, \theta_v)$, given by $\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}, \theta_v) - V(s_t, \theta_v)$, with k can vary by an upper bound t_{max} .

4.2 Proximal Policy Gradient

PPO is a new class of policy gradient techniques for RL, that alternates among sampling data via environment interaction and optimizing a clipped "surrogate" objective function L^{clip} [30], that represents the difference between the new policy after the most recent update $\pi_{\theta}(a | s)$ and the old policy before the most recent update $\pi_{\theta_k}(a | s)$. PPO features an objective function that allows many epochs of minibatch updates, whereas traditional policy gradient algorithms only conduct one gradient update per data sample. PPO outperforms other online policy gradient approaches, striking a favorable balance between sample complexity, simplicity, and wall-time. It has some of the features of trust region policy optimization, but they are far simpler to implement, more generic, and have superior sample complexity. The PPO update is determined using the following formula:

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s_t, a_t) \right) \quad (8)$$

Where ϵ refers to a constant hyperparameter.

5 Experimental Results

The proposed trading systems are tested on bitcoin historical data in www.bitcoincharts.com and www.kaggle.com platform. We considered bitcoin price in USD dollars from the 1st December 2016 to the 19th December 2020. the observation space is collected by using Open, Close, High, and Low prices. The in-sample period and the out-of-sample period are both taken into account. The first section offers information for the training and validation phases. The second section offers information for the trade stage.

DD- DQN, PPO, and A2C are used to learn three agents in the training stage. Then comes the validation stage, which involves validating the three agents using the Sharpe ratio (Sh) presented in Eq. 6 and changing critical parameters like learning rate, number of episodes, and so on. Finally, we analyze the profitability of each algorithm throughout the trading stage.

$$\text{Sh} = \frac{\bar{r}_p - r_f}{\sigma_p}$$

Where $\bar{r}_p$ denotes the expected asset return, r_f denotes the risk-free rate, and σ_p denotes the asset's standard deviation. In our validation stage, we also use the volatility metric to adjust risk aversion. A simple profit function pr_{profit} is also used to evaluate our results is calculated by subtracting the asset price value at time t (pr_t) from its at time $t-1$ value (pr_{t-1}).

The proposed algorithms are tested using the Colab GPU and Python implementation.

Table 1. The value of Sharpe ratios under time

Quarterly trading	DD-DQNs	PPO	A2C	Chosen Model
2018/01 – 2018/03	0.05	0.04	0.02	DD-DQNs
2018/04 – 2018/06	0.63	0.46	0.50	DD- DQNs
2018/07 – 2018/09	0.09	0.02	0.2	A2C
2018/10 – 2016/12	-0.11	0.22	-0.59	PPO
2019/01 – 2019/03	0.38	0.49	0.28	PPO
2019/04 – 2019/06	0.36	0.25	0.36	DD- DQNs
2019/07 – 2019/09	0.09	0.01	0.04	DD- DQNs
2019/10 – 2019/12	0.09	0.02	0.2	PPO
2020/01 – 2020/03	0.36	0.46	0.51	A2C
2020/04 – 2020/06	0.16	-0.28	0.36	A2C
2020/07 – 2020/09	0.10	0.09	0.13	A2C
2020/10 – 2020/12	0.08	0.08	0.16	A2C

As shown in Table 1, DD- DQNs have the greatest validation Sharpe ratio $Sh=0.05$ during 2017/10 to 2017/12, and we choose it to trade for the next period between 2018/01 to 2018/03. And we selected it because it has the best value of $Sh= 0.63$ from 2018/01 to 2018/03. We use also used to trade from the next quarter from 2018/04 to 2018/06. We can see that A2C has the best validation $Sh= 0.2$ from 2018/04 to 2018/06, then we can use A2C to trade for the following quarter through 2018/07 to 2018/09. We choose PPO to trade for the following quarter from 2018/10 to 2018/12 which has the best validation Sharpe ratio of -0.15 from 2018/07 to 2018/09. And so on.

The average return and Sharpe Ratio for different agent RL used in this work are calculated to compare the agent Performance. The result is illustrated in the following table.

Table 2. Performance evaluation comparison

(2018/01 – 2020/12)	Proposed Method	DD-DQNs	PPO	A2C
Profit reward function (average return (%))	12.84	12.6	11.75	09.84
Sharpe Ratio	1.41	1.39	1.25	1.36

We can observe from the results presented in table2, that the DD-DQNs agent is good at monitoring trends and acting well in generating greater returns; among the PPO and A2C agents, it has the greatest average return at 12.6 %. When faced with a rising market, the DD-DQNs are favored. In a bullish market, the PPO method performs similarly to DD-DQNs, but not as well. It can be used as a supplementary to DD-DQNs. The proposed method is based on a combination of the three agents, it is clear that the average return of the proposed approach reaches a higher value than other agents.

As shown in Table 2, the aggregate strategy has a Sharpe ratio of 1.41, outperforming PPO, A2C, and DD-DQN, which have Sharpe ratios of 1.25, 1.36, and 1.39, respectively. As a result, our findings show that the proposed aggregate strategy can be used to create a trading strategy that outperforms the three individual algorithms.

6 Conclusion

In this study, we investigated the power of applying Dueling Double Deep Q-Networks, Proximal Policy Optimization, and Advantage Actor-Critic algorithms to learn a cryptocurrency trading strategy. Over roughly two years, these strategies are compared. Based on Sharpe ratio and profit reward functions to evaluate the performance of different methods. The proposed approach is made by combining the previous approaches. The evaluation framework showed that our technique is the most profitable approach for cryptocurrency trading.

Future work is attractive for investigating more advanced models, dealing with larger-scale data, we can also explore other features for the state space, such as adding advanced transaction costs and a liquidity model

References

1. Iftikhar Ahmad, Muhammad Ovais Ahmad, Mohammed A Alqarni, Abdulwahab Ali Almazroi, and Muhammad Imran Khan Khalil, *Using algorithmic trading to analyze short term profitability of bitcoin*, PeerJ Computer Science **7** (2021), e337.
2. Bouchra El Akraoui and Cherki Daoui, *Deep learning for medical image segmentation*, International Conference on Business Intelligence, Springer, 2021, pp. 294–303.
3. Laura Alessandretti, Abeer ElBahrawy, Luca Maria Aiello, and Andrea Baronchelli, *Anticipating cryptocurrency prices using machine learning*, Complexity **2018** (2018).
4. Andrés Arévalo, Jaime Niño, German Hernández, and Javier Sandoval, *High-frequency trading strategy based on deep neural networks*, International conference on intelligent computing, Springer, 2016, pp. 424–436.
5. Andrés Ricardo Arévalo Murillo, *High-frequency trading strategy based on deep neural networks*, Ingeniería de Sistemas (2019).
6. Akhil Raj Azhikodan, Anvitha GK Bhat, and Mamatha V Jadhav, *Stock trading bot using deep reinforcement learning*, Innovations in Computer Science and Engineering, Springer, 2019, pp. 41–49.
7. D Britz, *Introduction to learning to trade with reinforcement learning*, URL <http://http://www.wildml.com/2018/02/introduction-to-learning-to-trade-with-reinforcement-learning> (2018).
8. Alain P Chaboud, Benjamin Chiquoine, Erik Hjalmarsson, and Clara Vega, *Rise of the machines: Algorithmic trading in the foreign exchange market*, The Journal of Finance **69** (2014), no. 5, 2045–2084.
9. Ernest P Chan, *Quantitative trading: how to build your own algorithmic trading business*, John Wiley & Sons, 2021.
10. Stuart Colianni, Stephanie Rosales, and Michael Signorotti, *Algorithmic trading of cryptocurrency based on twitter sentiment analysis*, CS229 Project **1** (2015), no. 5.
11. Rajashree Dash and Pradipta Kishore Dash, *A hybrid stock trading framework integrating technical analysis with machine learning techniques*, The Journal of Finance and Data Science **2** (2016), no. 1, 42–57.
12. Jeffrey Goldkamp and Mohammad Dehghanimohammadabadi, *Evolutionary multi-objective optimization for multivariate pairs trading*, Expert Systems with Applications **135** (2019), 113–128.
13. Yifeng Guo, Xingyu Fu, Yuyan Shi, and Mingwen Liu, *Robust log-optimal strategy with reinforcement learning*, arXiv preprint arXiv:1805.00205 (2018).
14. Kareem Hegazy and Samuel Mumford, *Comparative automated bitcoin trading strategies*, CS229 Project **27** (2016).
15. Chien Yi Huang, *Financial trading as a game: A deep reinforcement learning approach*, arXiv preprint arXiv:1807.02787 (2018).
16. Fabian Isensee, Paul F Jaeger, Simon AA Kohl, Jens Petersen, and Klaus H Maier-Hein, *nnu-net: a self-configuring method for deep learning-based biomedical image segmentation*, Nature methods **18** (2021), no. 2, 203–211.

17. Huisu Jang and Jaewook Lee, *An empirical study on modeling and prediction of bitcoin prices with bayesian neural networks based on blockchain information*, Ieee Access **6** (2017), 5427–5437.
18. Zhengyao Jiang and Jinjun Liang, *Cryptocurrency portfolio management with deep reinforcement learning*, 2017 Intelligent Systems Conference (IntelliSys), IEEE, 2017, pp. 905–913.
19. Youngmin Kim, Wonbin Ahn, Kyong Joo Oh, and David Enke, *An intelligent hybrid trading system for discovering trading rules for the futures market using rough sets and genetic algorithms*, Applied Soft Computing **55** (2017), 127–140.
20. Giorgio Lucarelli and Matteo Borrotti, *A deep reinforcement learning approach for automated cryptocurrency trading*, IFIP International Conference on Artificial Intelligence Applications and Innovations, Springer, 2019, pp. 247–258.
21. Wenchu Ma, Xuemin Tu, Bo Luo, and Guanghui Wang, *Semantic clustering based deduction learning for image recognition and classification*, Pattern Recognition **124** (2022), 108440.
22. Harry Markowitz, *Portfolio selection in the journal of finance vol. 7*, (1952).
23. Sean McNally, Jason Roche, and Simon Caton, *Predicting the price of bitcoin using machine learning*, 2018 26th euromicro international conference on parallel, distributed and network-based processing (PDP), IEEE, 2018, pp. 339–343.
24. Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu, *Asynchronous methods for deep reinforcement learning*, International conference on machine learning, PMLR, 2016, pp. 1928–1937.
25. Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller, *Playing atari with deep reinforcement learning*, arXiv preprint arXiv:1312.5602 (2013).
26. Amirhosein Mosavi, Yaser Faghan, Pedram Ghamisi, Puhong Duan, Sina Faizollahzadeh Ardabili, Ely Salwana, and Shahab S Band, *Comprehensive review of deep reinforcement learning methods and applications in economics*, Mathematics **8** (2020), no. 10, 1640.
27. John J Murphy, *Technical analysis of the financial markets: A comprehensive guide to trading methods and applications*, Penguin, 1999.
28. Mustafa ÖZYEŞİL, *Comparison of technical and fundamental analysis trading disciplines on portfolio performance: Short and long term backtest analysis on bursa istanbul national stock indices*, Journal of Contemporary Research in Business, Economics and Finance **3** (2021), no. 3, 128–143.
29. Otabek Sattarov, Azamjon Muminov, Cheol Won Lee, Hyun Kyu Kang, Ryumduck Oh, Junho Ahn, Hyung Jun Oh, and Heung Seok Jeon, *Recommending cryptocurrency trading points with deep reinforcement learning approach*, Applied Sciences **10** (2020), no. 4, 1506.
30. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, *Proximal policy optimization algorithms*, arXiv preprint arXiv:1707.06347 (2017).
31. Jack D Schwager, *A complete guide to the futures market: Technical analysis, trading systems, fundamental analysis, options, spreads, and trading principles*, John Wiley & Sons, 2017.
32. Thibaut Théate and Damien Ernst, *An application of deep reinforcement learning to algorithmic trading*, Expert Systems with Applications **173** (2021), 114632.
33. Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas, *Dueling network architectures for deep reinforcement learning*, International conference on machine learning, PMLR, 2016, pp. 1995–2003.

- 34. Christopher JCH Watkins and Peter Dayan, *Q-learning*, Machine learning **8** (1992), no. 3, 279–292.
- 35. Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid, *Deep reinforcement learning for automated stock trading: An ensemble strategy*, Proceedings of the First ACM International Conference on AI in Finance, 2020, pp. 1–8.
- 36. Pengfei Yu and Xuesong Yan, *Stock price prediction based on deep neural networks*, Neural Computing and Applications **32** (2020), no. 6, 1609–1628.
- 37. Dong Zhang, *Deep reinforcement learning in medical object detection and segmentation*, (2020).
- 38. Zihao Zhang, Stefan Zohren, and Stephen Roberts, *Deep learning for portfolio optimization*, The Journal of Financial Data Science **2** (2020), no. 4, 8–20.